

CSE 8803: Deep Learning for Text Data

Fall 2024

Homework 1

Due: Sep. 17, 2024

1. $\text{\LaTeX}$ Instructions

If you opt to use the provided $\text{\LaTeX}$ template, employ the command “`\answer{}`” to insert your answers. Placeholders are already set up for you. Simply place your answers inside the curly brackets.

2. Recurrent Neural Network

The idea behind RNNs is to make use of sequential information. The RNN structure is shown in Fig.1. The equations for $\mathbf{s}_t$ and $\mathbf{o}_t$ are:

$$\mathbf{s}_t = \sigma(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1}), \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{V}\mathbf{s}_t), \quad (1)$$

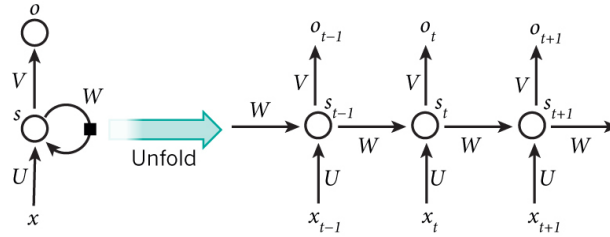


Figure 1. RNN structure (image from WildML)

where σ is the sigmoid function. The loss function we use is still cross entropy but across all of the time stamps:

$$L(\mathbf{y}, \hat{\mathbf{y}}) = -\frac{1}{N} \sum_t \mathbf{y}_t^\top \log \hat{\mathbf{y}}_t. \quad (2)$$

- (a) (5 points) Calculate $\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k}$, where $k < t$.

Answer:

$$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} = \prod_{i=k+1}^t \mathbf{W}^\top \cdot \sigma(\mathbf{U}\mathbf{x}_i + \mathbf{W}\mathbf{s}_{i-1}) \cdot (1 - \sigma(\mathbf{U}\mathbf{x}_i + \mathbf{W}\mathbf{s}_{i-1})), \text{ for } k \leq t$$

$$\mathbf{s}_t = \sigma(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1}),$$

Based on chain rule:

$$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} = \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_{t-1}} \cdot \frac{\partial \mathbf{s}_{t-1}}{\partial \mathbf{s}_{t-2}} \cdot \dots \cdot \frac{\partial \mathbf{s}_{k+1}}{\partial \mathbf{s}_k}$$

$$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_{t-1}} = \sigma'(\mathbf{U}\mathbf{x}_t + \mathbf{W}\mathbf{s}_{t-1}) \cdot \mathbf{W}^\top \rightarrow \frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} = \prod_{i=k+1}^t \sigma'(\mathbf{U}\mathbf{x}_i + \mathbf{W}\mathbf{s}_{i-1}) \cdot \mathbf{W}^\top$$

Derivative of sigmoid function:

$$\sigma'(x) = \sigma(x)(1 - \sigma(x))$$

[3]

Therefore,

$$\frac{\partial \mathbf{s}_t}{\partial \mathbf{s}_k} = \prod_{i=k+1}^t \mathbf{W}^\top \cdot \sigma(\mathbf{U}\mathbf{x}_i + \mathbf{W}\mathbf{s}_{i-1}) \cdot (1 - \sigma(\mathbf{U}\mathbf{x}_i + \mathbf{W}\mathbf{s}_{i-1})),$$

- (b) (5 points) Under what condition will the model suffer from the vanishing gradient issue? (Reference: On the Difficulty of Training Recurrent Neural Networks [7], ICML'13.)

Answer:

Because the hidden state k depends on hidden state t , output has relationship to multiplication of weight matrix W .

When W are less than one, the loss function's gradient will decrease to zero after multiplications, leading to difficulties of dependencies in long term between state k and t .

- (c) Long Short-Term Memory (LSTM) Networks is a special kind of RNN, capable of learning long-term dependencies. We define the LSTM units at each time step t to be a collection of vectors in $\mathcal{R}^d$: an input gate $\mathbf{i}_t$, a forget gate $\mathbf{f}_t$, and output gate $\mathbf{o}_t$, a memory cell $\mathbf{c}_t$ and a hidden state $\mathbf{h}_t$. d is the number of the LSTM units. The entries of the gating vectors $\mathbf{i}_t$, $\mathbf{f}_t$ and $\mathbf{o}_t$ are in $[0, 1]$. The LSTM transition equations are the

following:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i \mathbf{x}_t + \mathbf{U}_i \mathbf{h}_{t-1}), \quad (3)$$

$$\mathbf{f}_t = \sigma(\mathbf{W}_f \mathbf{x}_t + \mathbf{U}_f \mathbf{h}_{t-1}) \quad (4)$$

$$\mathbf{o}_t = \sigma(\mathbf{W}_o \mathbf{x}_t + \mathbf{U}_o \mathbf{h}_{t-1}), \quad (5)$$

$$\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c \mathbf{x}_t + \mathbf{U}_c \mathbf{h}_{t-1}), \quad (6)$$

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \quad (7)$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t), \quad (8)$$

Where $\odot$ denotes element-wise multiplication.

Judge True/False for the following and briefly justify your answer.

- i. (2 points) (True/False) If $\mathbf{x}_t$ is the $\mathbf{0}$ vector, then $\mathbf{h}_t = \mathbf{h}_{t-1}$.

False

Reason: $\mathbf{h}_t$ not only depend on $\mathbf{x}_t$, but also depend on $\mathbf{c}_{t-1}$ and $\mathbf{f}_t$, which are not necessarily zero.

- ii. (2 points) (True/False) If $\mathbf{f}_t$ is very small or zero, then error will not be back-propagated to earlier time steps.

True

Reason: This is the vanishing gradient issue.

- iii. (2 points) (True/False) The entries of $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ are non-negative.

True

Reason: $\mathbf{f}_t$, $\mathbf{i}_t$, and $\mathbf{o}_t$ are calculated from sigmoid function. Since sigmoid function maps input to output value ranging between 0 and 1, $\mathbf{f}_t$, $\mathbf{i}_t$, and $\mathbf{o}_t$ are all non-negative.

- iv. (2 points) (True/False) $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ can be viewed as probability distributions (entries sum to 1 and each entry is between 0 and 1).

False

Reason: The definition of probability distribution is the math function that gives the chance of outcome from an experience. $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ could not be considered as probability distribution just because they have similar range.

- v. (2 points) What is the difference between the c and h recurrent states in an LSTM recurrent unit?

$\mathbf{c}_t$: memory state, showing long-term information

$\mathbf{h}_t$: hidden stated, showing short-term information

3. Attention Mechanisms

Read the papers on attention mechanisms [2, 6] and answer the following questions:

- (a) (5 points) Briefly describe the problems faced by the original sequence-to-sequence models for text generation that attention mechanisms aim to solve.

- **Information Loss**

Fixed-length vector leads to loss of information in long sequence, leading to difficulty in learning dependency between long positions.

- **Long training time**

Architectures based on recurrent or convolutions layers have a longer training time and have limitations in batching process due to the sequential nature that leads to quick growing of operation numbers

- (b) (5 points) Bahdanau attention [2] and Luong attention [6] (specifically, the global Luong attention) are slightly different in terms of computing the attention weights (represented by α in [2] and a in [6], respectively). Please briefly describe the differences.

Note: Only discuss the difference in attention weight computation; including other differences will result in a deduction of points.

- **Bahdanau Attention**

Each weight is attention energy with the normalized results of all the steps. [8]

$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^{T-1} \exp(e_{ik})}$$

where $e_{ij} = a(s_{i-1}, h_j)$

- **Luong Attention**

Dot product computation

$$a_t = \text{softmax}(W_a h_t)$$

4. Self-Attention and Transformer

Read the famous paper “Attention is All You Need” by Vaswani et al. [9] and answer the questions below. The figures and equations referenced are based on this paper.

(a) (5 points) **Tokenization**

- i. Which tokenization method do Vaswani et al. employ?

Byte-pair encoding.

- ii. What advantages does this method offer?

Most common words are shown as single token while rare words are represented with plural tokens. This method will improve memory efficiency because it generates least amount of tokens. [5]

(b) (21 points) **Self-Attention**

Vaswani et al. introduce the self-attention mechanism using the concepts of query (Q), key (K), and value (V) as inputs to the attention function (refer to Figure 2 and Equation 1). If during model training the input sequence length is denoted as l_{input} , the output sequence length as l_{output} , the model dimensionality as d_{model} , and the number of attention heads as k :

- i. Describe the dimensions of the Q , K , and V matrices from Equation 1 for the t -th token in the *output* sequence during model training.

For the t -th token in the output sequence:

- dimensions of Q : $R^{d_k}, d_k = \frac{d_{\text{model}}}{h}$
- dimensions of K : $R^{d_k}, d_k = \frac{d_{\text{model}}}{h}$
- dimensions of V : $R^{d_v}, d_v = d_k = \frac{d_{\text{model}}}{h}$

- ii. What is the dimensionality of the attention output in Equation 1?

Answer: d_{model}

Reason:

l : Number of tokens in sequence, one dimension

d_{model} : Model hidden dimension

$\Rightarrow l \times d_{\text{model}}$

dimensionality of output is d_{model}

- iii. Will the dimensions of Q , K , and V change during inference? Please justify your answer.

Answer: **No, dimensions of Q, K, V will not change during inference**

Reason: In training phase, the dimensions of Q , K , and V do not change. Inference phase will only influence sequence length, but will not change the dimensions of model or attention head.

iv. What are the physical meanings of Q , K , and V in the context of the paper?

In the context of the paper:

- Q : Vector representation of current token, and it is used to retrieve information.
- K : Vector representation of all the tokens in the input sequence
- V : Vector representation of the information passed to next layers after getting the attention weights

v. Why are different weight matrices used for the Q and K vectors?

Using multi-head attention means to have separate sets of attention mechanisms, leading to model's ability to capture different relationships.

vi. Why is scaling applied in Attention mechanisms?

Scaling helps to set a range for dot products of query and key vector so that it does not grow exceedingly and ensures gradients are stable.

vii. What are the differences between LayerNorm and BatchNorm? Which of them is used in the Transformer? Why?

Difference:

- **BatchNorm**: Normalizes feature in a batch.
- **LayerNorm**: Normalize all features in a sample.[4]

Transformer uses LayerNorm.

LayerNorm computes mean and variance across the features. It fits transformers' token-level normalization to facilitate understanding relationships between tokens.

(c) (9 points) **Comparing Self-Attention Mechanisms**

i. Describe the differences between the self-attention mechanism in the Transformer's *encoder* and the Bahdanau/Luong attentions. Make your answer as comprehensive as possible.

Please refer to comparison table(table 1) below.

Feature	Self-Attention	Bahdanau/Luong Attention
Input	Single sequence	Two sequences (source & target)
Focus	Different parts of the same sequence,	Source sequence for target generation
Function	Encodes relationships in the same sequence	Encode relevant parts of the source
Attention Matrix	Token-to-token in same sequence	Target-to-source sequence

Table 1. Self-Attention and Bahdanau/Luong Attention Comparison

- ii. What is the purpose of including the self-attention mechanism in the *encoder*?

Let transformer model learn the context.

For example, word "window" has two meanings ("glass" and "time frame"), using self-attention in the transformer make the model understand its meaning based on the input context.[1]

- iii. What is the difference between the decoder's self-attention and the encoder's self-attention in a Transformer model? Decoder uses masked self-attention while encoder does not.

- (d) (3 points) **Positional Encoding**

What is the purpose of using *positional encoding* in the Transformer?

To provide positional information to transformer so that orders of tokens could be passed to next layer.

- (e) (6 points) **Input Sequence Length in Transformer**

- i. Is there a theoretical hard upper bound on the input sequence length for the Transformer architecture? If so, what is the cause? Please justify your answer.

No

Theoretically input could be arbitrary long. Because the batches will be the same length, and only depending on the longest sequence.

- ii. Is vanilla Transformer efficient in dealing with very long input sequences (*e.g.*, 4096 tokens)? Please justify your answer.

No

When input sequence is over 4000, the complexity is very large (increase quadratically), so the vanilla transformer is not efficient when dealing with very long input.

5. (26 points) Masked Multi-Head Attention

Please check the GitHub repository for this coding assignment: <https://github.com/Yinghao-Li/CSE8803DLT-24Fall-Assignment1-Public>

References

- [1]
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015.
- [3] GeeksforGeeks. Derivative of the sigmoid function. *GeeksforGeeks*, Jan 2023.
- [4] Florian June. Batchnorm and layernorm - florian june. *Medium*, May 2024.
- [5] Chetna Khanna. Byte-pair encoding: Subword-based tokenization. *Towards Data Science*, Sep 2024.
- [6] Thang Luong, Hieu Pham, and Christopher D. Manning. Effective approaches to attention-based neural machine translation. In Lluís Màrquez, Chris Callison-Burch, Jian Su, Daniele Pighin, and Yuval Marton, editors, *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing, EMNLP 2015, Lisbon, Portugal, September 17-21, 2015*, pages 1412–1421. The Association for Computational Linguistics, 2015.
- [7] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International conference on machine learning*, pages 1310–1318. PMLR, 2013.
- [8] spro. practical-pytorch/seq2seq-translation/seq2seq-translation.ipynb.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In Isabelle Guyon, Ulrike von Luxburg, Samy Bengio, Hanna M. Wallach, Rob Fergus, S. V. N. Vishwanathan, and Roman Garnett, editors, *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pages 5998–6008, 2017.